RentalPro

Digital Rental Management System

IT402 — DELIVERABLE 3

Final Capstone Project Report

Field	Details
Student Name	Abdur-Rahmaan Muhammad Ali
Student ID	ST10201821
Course Code	IT402 — IT Capstone II
Project Name	RentalPro — Digital Rental Management System
Deliverable	IT402 Deliverable 3 — Project Demo, Presentation and Final Report
Version	v1.0 — October 2026
Sections	Executive Summary · Project Overview & Objectives · Deliverables Summary · System Architecture · Key Features · Testing & Quality Assurance · Challenges & Lessons Learned · Conclusion
Appendix	Appendix — Project Statistics Summary
Accompanying Files	Project demo video (2:52) · Presentation video (18:04) · Presentation slides (PowerPoint and PDF)
Prior Deliverables	IT401 Deliverable 1 — SRS · IT401 Deliverable 2 — UML & Database Design · IT401 Deliverable 3 — Partially-Developed Application · IT402 Deliverable 1 — Complete Project Report · IT402 Deliverable 2 — Feature Completion

Table of Contents

1. Executive Summary....3

2. Project Overview & Objectives....3

2.1 Objectives....4

3. Deliverables Summary....4

3.1 IT401 Deliverable 3 — Partially-Developed Application....5

3.2 IT402 Deliverable 1 — Complete Project Report....5

3.3 IT402 Deliverable 2 — 100% Feature Completion....5

3.4 Extended Scope — The Billing Engine....6

3.5 Extended Scope — Usability, Localisation & Harare Demo Data....6

3.6 Extended Scope — Full Arabic & Production Hardening....7

4. System Architecture Summary....8

5. Key Features Delivered....9

5.1 By Role....9

5.2 IT402 Deliverable 2 Highlights....9

5.3 Billing Engine Highlights....9

5.4 Usability & Localisation Highlights....10

5.5 Full Arabic & Hardening Highlights....10

6. Testing & Quality Assurance Results....10

6.1 What the Suite Verifies....10

7. Challenges & Lessons Learned....11

8. Conclusion & Recommendations for Future Work....12

8.1 Recommendations Beyond IT402....13

Appendix: Project Statistics Summary....14

1. Executive Summary

RentalPro is a Digital Rental Management System built to replace the spreadsheets, paper leases, and manual reminders that independent landlords and small property managers still rely on. The project was carried out across two courses and five prior deliverables — a Software Requirements Specification (IT401 Deliverable 1), a UML and database design (IT401 Deliverable 2), a working prototype application (IT401 Deliverable 3), a complete seven-chapter project report (IT402 Deliverable 1), and, closing every remaining gap against the original requirements, 100% feature completion (IT402 Deliverable 2). IT402 Deliverable 3 — a project demonstration video, a presentation video, and this final capstone report — closes out the formal capstone submission. Beyond the original requirements, the system has also been extended with a complete billing engine (Section 3.4): eight additional capabilities covering charges, metering, prorated and variable rent, automated billing, statements, receipts, and expense reporting, all built additively and fully tested. It has also gained usability and localisation work (Section 3.5): a property map, a redesigned interface with Harare photography, English, Shona and Arabic interface languages chosen from a language menu (with a right-to-left layout for Arabic), and a Harare-based demonstration portfolio. Most recently (Section 3.6), Arabic was extended to every screen, message, email, SMS and document, every amount now shows its currency, and the system was hardened for production with security headers, a required secret key, database migrations and optional cloud file storage.

The finished system is a Flask web application backed by a database of 18 tables (13 from the original design plus 5 added for the billing engine), organized into 10 route blueprints and five role-scoped user experiences (Property Owner, Property Manager, Tenant, Maintenance Staff, and Administrator). Every one of the 25 use cases defined in the original requirements has a corresponding route or scheduled background job, a user interface, and at least one automated test. The application is verified by a 305-test automated suite (all 305 passing on the latest run; one scheduler test self-skips on dates near month-end) with zero failures and 86% statement coverage, and it runs entirely offline — no paid external account (email, SMS, or payment provider) is required to build, run, or test it.

The guiding principle across this project has been that a claim in the documentation should be a fact about the running system, not a target for it. Every number in this report — 25 use cases, 18 tables, 305 tests, 86% coverage — was produced by actually running the code.

2. Project Overview & Objectives

Small landlords and property managers commonly run their business without dedicated software: rent, lease, and maintenance data live across spreadsheets and email threads, nothing proactively flags a missed payment or an expiring lease, cash payments and verbal maintenance requests leave no accountability trail, and tenants have no way to self-serve for a balance, a receipt, or a repair update. RentalPro was scoped to solve these problems for five kinds of users on one platform.

2.1 Objectives

- **Centralize property & lease data:** one record per property, unit, tenant, and lease, replacing duplicate spreadsheets.
- **Automate the reminders that get missed:** background jobs alert on lease expiry, rent due, overdue rent, and overdue maintenance without relying on a human to remember.
- **Give every role a role-scoped view:** role-based access control combined with ownership checks, so one Property Owner can never see another Owner's portfolio.
- **Make every record auditable:** rent payments are immutable and every state-changing action writes to an append-only audit log.
- **Ship it fully tested, not just working:** 100% of the SRS's functional requirements implemented and covered by an automated test — the standard this report verifies throughout.

3. Deliverables Summary

The project was delivered incrementally across six deliverables in two courses, IT401 and IT402, each verified against the actual, running state of the code rather than aspirational targets.

Deliverable	What It Delivered	Status
IT401 D1 — SRS	25 use cases across 6 functional areas: auth/RBAC, property & unit management, tenant lifecycle, rent & payments, maintenance, and reporting	Complete
IT401 D2 — UML & DB Design	UML class model and a 13-table normalized relational schema with UUID foreign keys, carried through to the build unchanged	Complete
IT401 D3 — Partially-Developed Application	A working Flask application implementing the SRS, with an initial automated test suite (93 tests, 77% coverage); its report records that build as submitted	Complete
IT402 D1 — Project Report	Complete 7-chapter report (Introduction, OOA, OOD, Database Design, Software Development, Testing, Conclusion) plus Appendices A & B, as submitted, with a closing appendix on later changes	Complete
IT402 D2 — 100% Feature Completion	11 remaining SRS gaps closed: unit editing, photo uploads, tenant ID and lease documents, late fees, tenant blacklisting, admin console, staff self-service, CSV/Excel export — all covered by a dedicated 31-test module	Complete
IT402 D3 — Demo, Presentation & Final Report	Recorded application walkthrough, a presentation video with full narration script, and this final capstone report	Complete
Extended Scope — Billing Engine	8 further billing capabilities beyond the original SRS: charges, metering, pro rata rent, variable rent, auto billing, statements, receipts, expenses — 79 dedicated tests	Complete
Extended Scope — Usability & Localisation	Property map, a redesigned top-bar interface with Harare photo backgrounds, English, Shona and Arabic interface languages with a right-to-left layout for Arabic, tenant-friendly unit labels, a logged-in redirect fix, and Harare demo data — 27 dedicated tests	Complete

Deliverable	What It Delivered	Status
Extended Scope — Full Arabic & Hardening	Arabic across every screen, message, email, SMS and PDF; currency on every amount; security headers and a required secret key; Alembic migrations; optional S3 file storage; deployment to Render	Complete

3.1 IT401 Deliverable 3 — Partially-Developed Application

IT401 Deliverable 3 turned the IT401 Deliverable 2 design into a working Flask application: 13 SQLAlchemy models, RBAC- and IDOR-protected routes, APScheduler background jobs, and PDF/notification/payment services with an offline dev-mode fallback so the entire application could be demonstrated and tested without a paid external account. Its report stands exactly as submitted, recording that build: 93 tests at 77% coverage.

3.2 IT402 Deliverable 1 — Complete Project Report

IT402 Deliverable 1 documented the system in full: seven chapters covering requirements analysis, object-oriented analysis and design, database design, the software development process, testing methodology and results, and a conclusion, plus an Installation Guide and User Manual. Like IT401 Deliverable 3, its main chapters keep the figures of that build (93 tests, 77% coverage); a closing appendix (Appendix C) summarises every change since, so the report never mixes two points in time.

3.3 IT402 Deliverable 2 — 100% Feature Completion

An audit against the original SRS after the IT401 Deliverable 3 prototype identified eleven real gaps (listed in full in the IT402 Deliverable 2 report). Each was closed and covered by its own automated test:

- Editing an existing unit’s details and status, which could previously only be created.
- Property (≤10), unit (≤5), and maintenance (≤3) photo uploads, validated by file content (magic bytes), not extension.
- Configurable per-property late fees (fixed or percentage), applied automatically once a lease’s rent is overdue.
- Tenant blacklisting, blocking a future lease without affecting an existing one.
- Tenant ID document upload and secure serving.
- Lease document download, for management and for the tenant themselves.
- An administrator console for user and role management, with a safeguard preventing removal of the last active Owner.
- Self-service Maintenance Staff account creation by an Owner, Manager, or Admin.
- One-click CSV export of a lease’s rent payment history.
- A two-sheet Excel (.xlsx) portfolio report export.

The IT402 Deliverable 2 report later traced all 44 functional requirements individually and found two only partly met — the full tenant history of each unit (FR-015) and a maintenance summary of open, in-progress and closed requests per property per period (FR-036) — and one, the interface languages (FR-

040), not yet built. All three were completed in September 2026: a History page on every unit and a Maintenance Summary on the Reports page, with 13 new tests, and the language menu described in Section 3.5. All 44 requirements are now met, two of them (FR-021 overdue notices and FR-044 welcome emails) deliberately in a different way from the SRS wording.

3.4 Extended Scope — The Billing Engine

The project was also extended with a complete billing engine — eight further capabilities going beyond the original SRS, each designed additively so the existing 124-test suite kept passing unmodified, and each covered by its own new test module (79 tests in total):

- Operational & sundry charges: one-off costs beyond rent — cleaning, call-outs — billed straight to a lease and reflected in its balance.
- Electricity metering: a configurable per-property rate plus manually captured meter readings; consumption since the last reading bills itself as a charge automatically.
- Pro rata rent: a lease's first month prorates to a daily rate when a tenant moves in mid-month, instead of billing a full month.
- Variable / escalating rent: a scheduled rent revision takes effect from a chosen date forward — every already-elapsed billing period stays billed at the rate in effect when it was due.
- Automated recurring billing: a fifth scheduler job bills every elapsed period on every active lease exactly once, catching up on any missed days and never double-billing a period already invoiced.
- Account statements by period: an opening balance, in-period activity, and a closing balance for any date range, plus full tenant self-service access to their own statement and PDF.
- Payment receipts: every payment now has a real, downloadable PDF receipt — tenant, unit, amount, method, and the balance at the moment of payment — not just a receipt number.
- Property expenses & net income: owner-side operating costs (tax, insurance, management fees, general upkeep) are now tracked per property and feed a true net income estimate, alongside maintenance cost, in both the on-screen and exported reports.

3.5 Extended Scope — Usability, Localisation & Harare Demo Data

The most recent round of work focused on how the system looks and reads to the people who would actually use it — landlords, managers and tenants in Harare — rather than on new business rules. Each change was made additively, behind the existing tests, with its own new tests:

- Property map: every property can carry map coordinates, set by clicking a map on the Add/Edit Property forms. A property's detail page shows its pin, and a portfolio map (/properties/map) plots every located property with a popup linking back to it. Leaflet is vendored into the app rather than loaded from a CDN (8 tests).
- Redesigned interface: the sidebar was replaced by a horizontal top navigation bar with a soft blue-to-burgundy gradient. The login page is a full-screen photograph of the Harare skyline with a translucent sign-in card, and each section of the app has its own Harare photograph behind it — the skyline for the dashboard, reports and admin pages, an aerial view for properties, and a

night street scene for tenants, rent, maintenance and the tenant portal. Cards and tables stay solid white and page titles are white with a soft shadow, so content stays readable.

- English, Shona and Arabic: a language button in the top bar and on the login page shows the current language and opens a list of English, ChiShona and Arabic. A small built-in translator (backend/i18n.py) looks each interface phrase up in the Shona or Arabic table (98 phrases each at the time) and falls back to English for anything not yet translated. Arabic pages are laid out right to left: the navigation starts from the right and tables read from right to left, and the stylesheet aligns text to the start of the line so one stylesheet serves both directions. The choice is kept in the session and survives logging in and out. Navigation, login, the dashboards, the main list pages, the tenant portal and status badges were translated in this round, and Arabic was completed in Section 3.6; data — property names, addresses, suburbs, cities and people's names — is never translated, and a test enforces that in both languages (14 tests).
- Tenant-friendly unit labels: the tenant dashboard and Pay Rent page name the property and unit (for example “Avondale Heights · Unit 101”) instead of an internal unit code (1 test).
- Logged-in redirect fix: opening the home page, the login page, or a password-reset or invitation link while already signed in used to show an empty page; it now takes the user to their own dashboard, portal or work queue (4 tests).
- Harare demonstration data: the seed data now describes a Harare portfolio — five properties in Avondale, the CBD, Eastlea and Vainona, each with map coordinates; 16 units; nine tenants with Zimbabwean names, national IDs and +263 phone numbers; a payment history for every lease with three tenants in arrears; and the administrator account in the author's name.

The SRS originally described multi-language support as Arabic and English. Both are now implemented, together with Shona: Shona, the most widely spoken language in Harare, serves the landlords, managers and tenants the system was built for, and Arabic serves Arabic-speaking residents and property owners in Zimbabwe. The translation mechanism is language-neutral, so a further language, such as Ndebele, only needs a new phrase table. The Shona and Arabic wording has not yet been reviewed by native speakers.

3.6 Extended Scope — Full Arabic & Production Hardening

The final round of work finished the language support and prepared the system for a live deployment, which followed on Render. As before, every change was additive and covered by new tests — 54 for the language and hardening work, across six test modules, and 7 for the Render deployment:

- Arabic on every screen: every page, form option, flash message and validation error now goes through the translator — over 600 phrases. Stored codes (payment methods, expense categories, unit types, roles) are shown as words in the reader's language, and a test fails if any text shown or sent by the app has no Arabic entry.
- Emails and SMS in each user's language: each user's language choice is saved to their account (a new users.language column), follows them to any browser, and is used for every email and SMS — including those sent by the nightly scheduler jobs, where there is no browser session to take it from.

- Arabic PDFs and spreadsheets: statements, receipts and reports are written in the reader's language. ReportLab draws text exactly as given, so Arabic PDFs use the bundled DejaVu Sans font, join the letters into their connected forms (arabic-reshaper), put each line into right-to-left order (python-bidi), mirror the tables, and keep dates, codes and amounts in their own order. Arabic Excel sheets open right to left, and the CSV export carries a byte-order mark so Excel reads Arabic headings.
- Currency everywhere: every amount — on screens, in messages, emails and PDFs — is shown the same way, as “USD 1,250.00”, using the property's own currency; portfolio totals show a currency only when every property shares it.
- Security headers and a required secret key: every response carries a strict Content-Security-Policy (scripts and styles from this site only, with all inline code moved into static files), X-Frame-Options, nosniff, a strict Referrer-Policy and HSTS over HTTPS. The session cookie is HttpOnly and SameSite, and HTTPS-only when configured. The application refuses to start with a missing or published SECRET_KEY, which would otherwise let anyone forge a login. An open redirect in the login and language-switch links and four colour-contrast failures were also fixed.
- Database migrations: Alembic (Flask-Migrate) now manages the schema. flask create-db applies pending migrations, and a database created earlier by db.create_all() is recognised and adopted with its data; a test fails if the models and the migrations ever describe different schemas.
- Cloud file storage: uploads can be kept in an S3-compatible bucket (AWS S3, Cloudflare R2, MinIO) instead of the server's disk, so they survive redeploys. Files stay private — they are streamed through the app after its permission checks, never exposed by a public link.
- Deployment to Render: render.yaml deploys the Docker image and a PostgreSQL database to Render, served over HTTPS. Behind the host's proxy the app recognises HTTPS and each visitor's own address (TRUST_FORWARDED_PROTO and CLIENT_IP_HEADER, both off by default, since a directly reached app could otherwise be fooled by forged headers), database URLs name the installed PostgreSQL driver, and only one Gunicorn worker runs the background jobs, so no notice is sent twice.

4. System Architecture Summary

RentalPro is a layered Flask application — a deliberate simplification of IT401 Deliverable 2’s more ambitious microservice-style sketch, scoped to what one developer could build and fully verify.

Layer	Contents
Presentation	40 Jinja2 templates behind a role-aware top navigation bar with an English/Shona/Arabic language menu, plus a shared design system in static/css/style.css and all page scripts in static/js (no inline code)
Application	10 Flask Blueprints: auth, dashboard, properties, tenants, rent, maintenance, admin, reports, portal, api
Domain / Service	security.py (RBAC, IDOR, validation, audit log), web_security.py (security headers, secret-key check), i18n.py (English/Shona/Arabic translation, right-to-left for Arabic, currency formatting), scheduler_jobs.py (5 background jobs), services/ (notifications, payments, pdf, excel, reports, storage)

Layer	Contents
Data	18 SQLAlchemy models with String(36) UUID primary keys, naive UTC datetimes, and business logic (balances, numbering, occupancy, billing) on the model itself; the schema is managed by Alembic migrations
Infrastructure	PostgreSQL (production) / SQLite (dev & test), Redis (caching, rate limiting), Docker Compose, uploads on local disk or in an S3-compatible bucket, with a dev-mode fallback for SendGrid, Twilio, and Stripe

Two documented, deliberate substitutions from the IT401 Deliverable 2 design carried through to the build: reportlab in place of WeasyPrint for PDF export (pure Python, no native system libraries), and String(36) UUID primary keys in place of a PostgreSQL-native UUID column type, so the identical schema runs unmodified on SQLite in development and test and PostgreSQL in production.

5. Key Features Delivered

5.1 By Role

Role	Core Capabilities
Property Owner	Create & edit properties and units, upload photos, configure late fees, full portfolio reporting
Property Manager	Manage properties an Owner has added, onboard and blacklist tenants, assign and oversee maintenance work
Tenant	Self-service portal: view lease and balance, pay rent online, submit maintenance requests with photos, download statements
Maintenance Staff	A personal work queue only — assigned tickets, status updates, completion photos; no visibility into properties, tenants, or rent
Administrator	Full data visibility across the portfolio, plus a dedicated console for user and role management and staff account creation

5.2 IT402 Deliverable 2 Highlights

The eleven gaps closed in IT402 Deliverable 2 (detailed in Section 3.3) brought the application to 100% coverage of the SRS's functional requirements — photo uploads across three entity types, configurable late fees, tenant blacklisting, ID document management, an admin console, staff self-service accounts, and CSV/Excel export.

5.3 Billing Engine Highlights

The eight billing capabilities added beyond the SRS (detailed in Section 3.4) turned RentalPro from a system that could only bill flat monthly rent into one with a genuine billing engine: ad-hoc charges, metered utilities, proration, mid-lease rent changes, hands-free recurring invoicing, real bank-style account statements, downloadable payment receipts, and owner-side expense tracking feeding a true

net income figure — all additive to the existing balance calculation, and all reachable from the same role-scoped screens described in Section 5.1.

5.4 Usability & Localisation Highlights

The usability and localisation work (Section 3.5) changed what every role sees rather than what the system can do: a map of the whole portfolio, a cleaner top-bar layout with Harare photography, and a choice of English, Shona or Arabic, so a Harare tenant can check a balance or pay rent in Shona, and an Arabic-speaking tenant can do the same in Arabic, laid out right to left. It sits on the same role-scoped routes described in Section 5.1, and demonstrating the system with a realistic Harare portfolio makes every dashboard, report and rent tracker figure meaningful.

5.5 Full Arabic & Hardening Highlights

The final round (Section 3.6) made the Arabic interface complete — an Arabic-speaking tenant now sees every screen, message, email, SMS and PDF in Arabic — and closed the gap between a working system and a deployable one: strict security headers, a required secret key, schema migrations that protect live data, and file storage that survives a redeploy.

6. Testing & Quality Assurance Results

RentalPro is verified with pytest against a fresh in-memory SQLite database per test, using Flask's own test client to issue real HTTP requests through the full middleware stack (session handling, CSRF, RBAC, and the ORM) rather than calling view functions in isolation.

Metric	Result
Tests collected	305
Tests passed	305
Tests skipped	0 on the latest run (one scheduler test self-skips on dates near month-end, where due-day clipping could shift the asserted date by a day)
Tests failed	0
Statement coverage (pytest --cov=backend)	86%
Roles exercised by the auth test suite	5 (Admin, Owner, Manager, Staff, Tenant)
Use cases with at least one covering test	25 / 25

6.1 What the Suite Verifies

- Business logic: password hashing, occupancy rate, balance calculations that never go negative, due-date math at calendar edge cases (mid-month starts, day-31-in-February clipping), receipt/ticket number uniqueness under rapid creation.

- Security: RBAC across all 5 roles against every management route; IDOR protection (a second Property Owner cannot view the first Owner's property); account lockout after repeated failed logins.
- End-to-end flows: property/unit creation, tenant onboarding and invitation, lease upload with PDF validation, rent payment recording, the full maintenance lifecycle, and report generation.
- Background jobs: all five scheduler jobs (lease expiry, rent due, automated recurring billing, overdue rent, maintenance escalation) invoked directly and asserted against their exact trigger thresholds.
- IT402 Deliverable 2 features: every one of the eleven closed gaps, covered by a dedicated 31-test module (tests/test_deliverable5.py).
- Billing engine: every one of the eight capabilities from Section 3.4, covered by its own dedicated test module — 79 tests in total, the single largest block of the suite.
- Usability & localisation: the three languages (English by default, switching to Shona and to Arabic, right-to-left for Arabic only, the choice surviving login, the language menu, no open redirect, placeholders kept, Arabic covering every Shona phrase, place names never translated), the property map, the logged-in redirect and the tenant unit labels — 27 tests.
- Requirement completion: the tenant history per unit (FR-015) and the maintenance summary per period (FR-036), including access for each role and date-range handling — 13 tests (tests/test_unit_history_and_maintenance_summary.py).
- Full Arabic & hardening: Arabic coverage of every phrase the app shows or sends, emails and SMS in the recipient's language, Arabic PDFs, Excel and CSV, currency formatting, security headers, the secret-key check, cookie flags, the translated rate-limit page, migrations matching the models, database adoption, and S3 storage — 54 tests (tests/test_translated_documents.py, test_web_security.py, test_migrations.py, test_money.py, test_storage.py and additions to test_i18n.py).

Coverage is concentrated in models.py (99%), security.py (92%), the reports service (96%), and the new web-security module (100%) — the modules carrying the business rules this project cares most about proving correct. The lowest-coverage files are thin route handlers whose uncovered branches are error-message formatting, and seed.py (demo-data tooling, intentionally outside the suite's scope).

7. Challenges & Lessons Learned

Challenge	How It Was Addressed
Reconciling ambitious early design documents with what one developer could build and verify in the available time	Simplified IT401 Deliverable 2's microservice-style package diagram to a layered monolith (Section 4); when IT401 Deliverable 3's originally-stated test and coverage numbers were found to be overstated, corrected the document to the real, verified numbers rather than leaving the discrepancy in place
Getting recurring, calendar-based billing correct at month boundaries (a lease starting on the 31st, a due day that doesn't exist in February)	Wrote the edge-case tests before trusting the implementation, catching clipping errors that would otherwise have shifted a due date by a day

Challenge	How It Was Addressed
Keeping the whole system — including payments and notifications — testable without any live external account	Built a dev-mode fallback pattern into every external service integration (SendGrid, Twilio, Stripe), which is also what lets the demo video and this whole project run fully offline
Keeping documentation honest as the system evolved past its initial prototype	Every figure is produced by running the code. IT401 Deliverable 3 stands exactly as submitted; IT402 Deliverable 1 keeps the figures of its own build in its main chapters, with one clearly marked closing appendix on later changes; and later reports describe each change in its own section, so no report mixes two points in time
Extending a fully-tested, shipped system with eight major billing features without breaking any of it	Every new capability (charges, metering, pro rata, revisions, auto billing, statements, receipts, expenses) was designed additively — new tables and nullable/defaulted fields only, so an existing lease with none of them set behaves exactly as it did before — and the full pre-existing suite was re-run after every single change to confirm zero regressions before moving to the next
Adding two more languages late in the project — one of them written right to left — without a translation library and without translating the data by mistake	Wrapped only interface text in a small _() helper with an English fallback, so the app works at every stage of translation; switched Arabic pages to right to left and aligned text to the start of the line so one stylesheet serves both directions; kept names and places as plain data, and added a test that fails if a Harare place name is ever changed or given a Shona or Arabic entry
Right-to-left Arabic in PDFs, where the PDF library draws text exactly as given	Joined the letters with arabic-reshaper and reordered each line with python-bidi, wrapping long lines before reordering so paragraphs still read top to bottom; isolated dates, codes and amounts so they keep their order; mirrored brackets by hand after finding the bidi library reorders text without mirroring them; and checked the rendered pages visually, not just by test

The recurring lesson across all six deliverables is the same one stated in the Executive Summary: documentation drifts from a real system unless it is actively re-verified against that system's current behavior, not just written once and trusted.

8. Conclusion & Recommendations for Future Work

RentalPro was carried from a requirements document, through a design, to a working prototype, a full project report, 100% feature completion, a demonstration and presentation package, an extended billing engine, a localised, Harare-focused interface, full Arabic and production hardening, and finally deployment to Render. Every claim in this document is backed by a running, tested system: 25 of 25 use cases implemented, each with a route and a covering test; the original 13-table database design implemented exactly as specified, since grown to 18 tables; all 305 automated tests passing with zero failures at 86% statement coverage; eleven feature gaps closed in IT402 Deliverable 2, and the three requirements it left open completed afterwards; eight further billing capabilities added afterward; an English, Shona and Arabic interface, with Arabic reaching every screen, message and document; security headers, schema migrations, cloud file storage and a Render deployment; and a property map with Harare demonstration data. The application runs — and its test suite passes — using only Docker or a local Python virtual environment, with zero paid external accounts required.

8.1 Recommendations Beyond IT402

- **Native-speaker review: have Arabic and Shona speakers check the wording, and extend Shona beyond the main screens it covers today (untranslated Shona text falls back to English).**
- **Mobile-optimized tenant portal:** tenants are the role most likely to use the system from a phone, and the current UI is not yet tuned for that.
- **Live production use: render.yaml deploys the app and a PostgreSQL database to Render over HTTPS; running it with real tenants still needs uploads in an S3 bucket (the server's own disk does not survive a redeploy) and a database plan that keeps data long term.**
- **Live third-party verification:** end-to-end tests against real Stripe, SendGrid, and Twilio sandbox accounts, in addition to the existing offline test suite.
- **Two-factor sign-in for the Owner, Manager and Administrator roles, which can see the most data.**

The discipline applied throughout this project — that a claim in the documentation should be a fact about the running system, not a target for it — is, in the author’s view, as much a deliverable of this capstone as the application itself.

Appendix: Project Statistics Summary

Statistic	Value
Use cases (SRS)	25
Database tables	18 (13 original + 5 for the billing engine)
Route blueprints	10
User roles	5
Jinja2 templates	40
Background scheduler jobs	5
Automated tests	305
Tests passing	305
Tests skipped	0
Test failures	0
Statement coverage	86%
IT402 Deliverable 2 feature gaps closed	11
IT402 Deliverable 2 dedicated tests	31
Billing engine features added	8
Billing engine dedicated tests	79
Usability & localisation dedicated tests	27
Tenant history & maintenance summary tests	13
Full Arabic & hardening dedicated tests	54
Render deployment dedicated tests	7
SRS functional requirements met	44 / 44 (FR-021 and FR-044 by design in a different way)
Interface languages	3 (English, Shona, Arabic — Arabic across every screen, message and document)
Demo portfolio	5 Harare properties, 16 units, 9 tenants
Deployment	Docker Compose (Flask/Gunicorn + PostgreSQL + Redis), GitHub Actions CI, Alembic migrations, optional S3 file storage, Render (render.yaml)